

Thiết kế và kiểm chứng lõi IP mật mã Ascon-AEAD128 tích hợp chuẩn giao tiếp AMBA APB trên FPGA

Hồ Thiện Nhân, 23521073

Khoa Kỹ thuật máy tính, Trường Đại học Công Nghệ Thông tin

ThS. Tạ Trí Đức

Khoa Kỹ thuật máy tính, Trường Đại học Công Nghệ Thông tin

Tóm tắt—Đề án trình bày thiết kế và kiểm chứng một lõi IP phần cứng hiện thực thuật toán mã hóa xác thực hạng nhẹ Ascon-AEAD128 theo chuẩn NIST SP 800-232, với rate 128 bit, hoán vị trung gian p8 và biểu diễn little-endian. Lõi được đóng gói cùng giao diện AMBA APB slave để tích hợp vào hệ thống SoC. Thiết kế được xây dựng từ dưới lên bằng Verilog-2001 thuần, gồm các khối `ascon_sbox`, `ascon_linear`, `ascon_round`, `ascon_perm`, `ascon_aead_fsm` và `ascon_apb`. Việc kiểm chứng dựa trên mô hình tham chiếu Python, đối chiếu toàn bộ 1089 test vector chính thức của NIST. Bộ kiểm chứng còn bao gồm các trường hợp lật bit trên ciphertext, tag và dữ liệu liên kết, cùng một bộ kiểm tra giao thức APB viết thuần Verilog. Ba phương án vi kiến trúc, ứng với số vòng hoán vị mỗi chu kỳ (ROUNDS_PER_CYCLE) bằng 1, 2 và 4, được tổng hợp Out-of-Context và quét tần số hoạt động tối đa trên FPGA Artix-7 (xc7a35tcbg236-1). Kết quả cho thấy phương án hai vòng mỗi chu kỳ đạt hiệu suất diện tích cao nhất, 1.8897 Mbits/LUT, và là điểm tối ưu Pareto giữa diện tích và thông lượng. Thiết kế cũng được khảo sát trên ba dòng FPGA Xilinx 7-series gồm Artix-7, Kintex-7 và Virtex-7, đạt Fmax cao nhất 304.04 MHz trên Kintex-7. Toàn bộ testbench pass 100%, tổng hợp không suy ra Block RAM hay DSP, và không vi phạm timing sau khi đặt chỗ và đi dây, với WNS = +0.013 ns tại chu kỳ 5.5 ns. Lõi IP do đó đạt hiệu suất diện tích cạnh tranh so với các công trình hiện thực Ascon trên FPGA công bố gần đây, dù các công trình đối chiếu hiện thực biến thể thuật toán cũ hơn là Ascon-128 v1.2.

Từ khóa—Ascon-AEAD128; mật mã hạng nhẹ; NIST SP 800-232; AMBA APB; lõi IP phần cứng; FPGA; Verilog; PPA (diện tích – hiệu năng – công suất)

I. GIỚI THIỆU

Các thiết bị Internet of Things (IoT) và hệ thống nhúng ngày càng phổ biến trong những ứng dụng có ràng buộc nghiêm ngặt về diện tích mạch, công suất tiêu thụ và tài nguyên tính toán, tiêu biểu là cảm biến không dây, thẻ RFID thụ động và thiết bị đeo. Các thuật toán mã hóa xác thực tiêu chuẩn (Authenticated Encryption with Associated Data, viết tắt là AEAD) như AES-GCM tuy an toàn và đã được kiểm chứng rộng rãi, nhưng đòi hỏi tài nguyên phần cứng vượt quá khả năng của nhiều lớp thiết bị hạn chế nói trên. Từ đó nảy sinh nhu cầu cấp thiết về một họ thuật toán mật mã hạng nhẹ (lightweight cryptography): vẫn đảm bảo mức an toàn 128 bit, nhưng có chi phí hiện thực trên phần cứng và phần mềm thấp hơn đáng kể.

Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ (NIST) khởi động dự án chuẩn hóa mật mã hạng nhẹ (Lightweight Cryptography Standardization) từ năm 2015. Dự án trải qua nhiều vòng đánh giá công khai với sự tham gia của cộng đồng

mật mã học quốc tế, và đến tháng 2/2023 công bố Ascon là họ thuật toán được chọn. Tháng 8/2025, NIST ban hành chính thức đặc tả SP 800-232 [1]. Thuật toán mã hóa xác thực chính trong đặc tả này, Ascon-AEAD128, có một số khác biệt cấu trúc so với bản đề trình gốc Ascon-128 v1.2, khiến bản mã và tag của hai biến thể hoàn toàn khác nhau dù cùng tên gọi họ thuật toán. Chi tiết các khác biệt được trình bày ở mục II.A. Đây là chuẩn còn rất mới, nên các công trình FPGA công bố tính đến thời điểm thực hiện đề án hầu hết vẫn hiện thực biến thể v1.2 cũ hơn.

Đề án đặt mục tiêu thiết kế và kiểm chứng một lõi IP phần cứng hiện thực đúng đặc tả Ascon-AEAD128 theo NIST SP 800-232, đóng gói với giao diện chuẩn công nghiệp AMBA APB để có thể tích hợp trực tiếp vào một hệ thống SoC. Một ví dụ tích hợp điển hình là vi xử lý nhúng điều khiển ngoại vi mật mã thông qua bus APB. Song song với đó, đề án khảo sát đánh đổi giữa diện tích, hiệu năng và công suất (PPA) trên nhiều phương án vi kiến trúc và nhiều dòng FPGA khác nhau.

Tính đến thời điểm báo cáo, đề án đã hoàn thành các nội dung sau:

Mô hình tham chiếu Python pass 100% bộ test vector KAT chính thức của NIST, gồm 1089 vector.

Xây dựng và kiểm chứng RTL từ dưới lên bằng Verilog-2001 thuần, với ba phương án vi kiến trúc ứng với ROUNDS_PER_CYCLE bằng 1, 2 và 4.

Tổng hợp Out-of-Context và quét Fmax trên ba dòng FPGA Xilinx 7-series.

Đo công suất và kiểm tra timing tĩnh sau khi đặt chỗ và đi dây.

Phần II đến phần IV lần lượt trình bày lý thuyết thuật toán, thiết kế vi kiến trúc, kết quả kiểm chứng cùng số liệu PPA, và cuối cùng là các hạn chế và hướng phát triển hiện tại.

II. GIẢI PHÁP

A. Lý thuyết

Ascon là họ thuật toán mật mã hạng nhẹ được NIST chọn làm chuẩn mã hóa xác thực (AEAD) và bấm hạng nhẹ vào tháng 2/2023, và được đặc tả chính thức trong NIST SP 800-232 ban hành tháng 8/2025 [1]. Đề án hiện thực đúng biến thể Ascon-AEAD128 theo đặc tả này. Trạng thái nội bộ rộng 320 bit, biểu diễn thành năm từ 64 bit ký hiệu S0 đến S4. Hai từ S0 và S1, tổng cộng 128 bit, tạo thành phần rate trao đổi trực tiếp với dữ

liệu vào và ra; ba từ còn lại S2, S3 và S4, tổng cộng 192 bit, tạo thành phần capacity không lộ ra ngoài. Khóa (K), nonce (N) và tag (T) đều rộng 128 bit. Hoán vị khởi tạo và hoàn tất dùng 12 vòng (p12), hoán vị xử lý dữ liệu trung gian dùng 8 vòng (p8). Giá trị khởi tạo IV cố định bằng 0x00001000808c0001.

Cần nhấn mạnh rằng Ascon-AEAD128 theo SP 800-232 khác biệt về cấu trúc so với bản đề trình cuộc thi gốc Ascon-128 phiên bản v1.2. Bốn khác biệt chính gồm:

- Rate tăng từ 64 lên 128 bit.
- Hoán vị xử lý dữ liệu trung gian đổi từ p6 sang p8.
- Biểu diễn dữ liệu đổi từ big-endian sang little-endian.
- Giá trị khởi tạo IV khác nhau.

Hai biến thể vì vậy cho ra bản mã và tag hoàn toàn khác nhau, dù cùng mang tên gọi họ thuật toán quen thuộc là ASCON. Toàn bộ mô hình tham chiếu và RTL của đề án bám theo đúng SP 800-232. Mô hình tham chiếu Python được lấy làm chuẩn đối chiếu: khi kết quả của RTL và mô hình sai khác, RTL được coi là sai, trừ trường hợp chính mô hình không vượt qua bộ test vector KAT chính thức của NIST.

Một phiên AEAD hoàn chỉnh gồm bốn giai đoạn tuần tự, minh họa ở Hình 1:

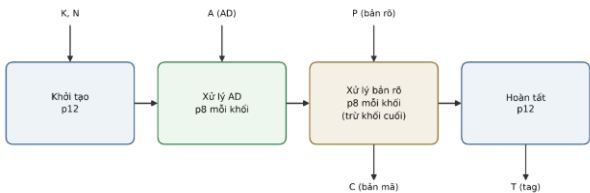
Khởi tạo: nạp IV, khóa và nonce vào trạng thái, sau đó chạy p12.

Xử lý dữ liệu liên kết (AD): mỗi khối được XOR vào phần rate rồi chạy p8. Giai đoạn này bị bỏ qua hoàn toàn nếu AD rỗng.

Xử lý bản rõ hoặc bản mã: mỗi khối được XOR vào phần rate rồi chạy p8, riêng khối cuối cùng không chạy p8.

Hoàn tất: XOR khóa vào phần capacity rồi chạy p12 lần cuối để sinh tag 128 bit.

Một trường hợp biên cần lưu ý: khi AD rỗng, lõi vẫn phải XOR bit phân tách miền vào S4 tại khối bản rõ đầu tiên. Đây là một trong những điểm dễ hiện thực sai nhất của đặc tả.



Hình 1 — Bốn giai đoạn xử lý của Ascon-AEAD128

Giao diện điều khiển lõi IP tuân theo chuẩn AMBA APB (Advanced Peripheral Bus) của Arm [2]. Đây là chuẩn bus ngoại vi đơn giản, băng thông thấp, phổ biến để kết nối các IP điều khiển và cấu hình vào vi xử lý trong SoC. Lõi IP hiện thực tập tín hiệu APB3 gồm psl, penable, pwrite, paddr, pwdata, prdata, pready và pslverr, là tập con của đặc tả APB5 trong tài liệu tham chiếu.

Mỗi giao dịch APB gồm hai pha là SETUP và ACCESS. Trong vai trò slave, lõi IP nhận các tín hiệu điều khiển từ master:

- psel — chọn slave
- penable — đánh dấu pha ACCESS
- pwrite — phân biệt thao tác ghi và đọc
- paddr, pwdata — địa chỉ và dữ liệu ghi, rộng 32 bit và trả về cho master:
- prdata — dữ liệu đọc, rộng 32 bit
- pready — báo sẵn sàng kết thúc giao dịch
- pslverr — báo lỗi truy cập

Lõi IP đóng vai trò APB slave, phôi ra một register map 32 bit theo từ tại các offset từ 0x00 đến 0x6C. Register map này cho phép vi xử lý ghi khóa, nonce và dữ liệu vào, đồng thời đọc dữ liệu ra và tag theo đúng trình tự thao tác của giao thức Ascon-AEAD128.

Đặc tả cũng quy định ba giới hạn an toàn mà lớp phần mềm điều khiển bắt buộc tuân thủ:

Nonce phải duy nhất với mỗi lần mã hóa dưới cùng một khóa.

Tổng dữ liệu xử lý dưới một khóa không vượt quá 2⁵⁴ byte.

Số lần giải mã thất bại dưới một khóa không vượt quá 2⁹⁶ lần.

Các giới hạn này nằm ngoài phạm vi kiểm chứng bằng RTL, nhưng được ghi rõ trong tài liệu tích hợp của lõi IP.

B. Thiết kế

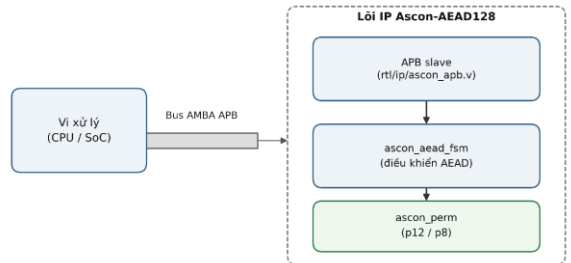
Lõi IP được tổ chức thành ba khối, xếp theo quan hệ sở hữu một chiều từ tầng cao xuống tầng thấp, không có tham chiếu ngược:

ascon_apb đóng vai trò APB slave và sở hữu một thực thể ascon_aead_fsm.

ascon_aead_fsm điều khiển toàn bộ luồng AEAD và sở hữu một thực thể ascon_perm.

ascon_perm thực hiện hoán vị p12 và p8.

Hình 2 minh họa cách lõi IP được tích hợp vào hệ thống. Một vi xử lý trong SoC giao tiếp với lõi qua bus AMBA APB, và bên trong lõi, ba khối nói trên xếp thành một chuỗi sở hữu tuần tự.



Hình 2 — Sơ đồ khối tích hợp lõi IP Ascon-AEAD128 vào hệ thống qua giao diện AMBA APB slave.

Vi kiến trúc cơ sở, ứng với ROUNDS_PER_CYCLE bằng 1 và minh họa ở Hình 3, lấy thanh ghi trạng thái 320 bit làm điểm

neo duy nhất của thiết kế. Mọi khối phía trên chỉ tính tổ hợp giá trị kế tiếp, không tự giữ trạng thái.

Khối ascon_round là mạch tổ hợp thuần túy, gồm ba thành phần:

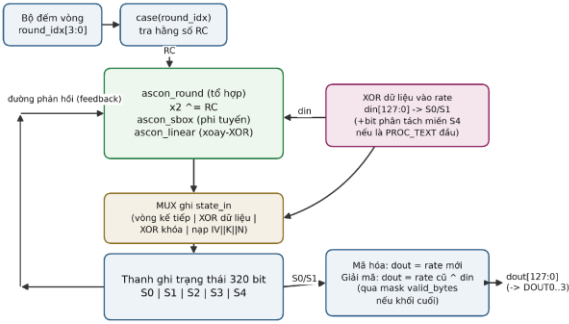
XOR hằng số vòng.

Lớp phi tuyến ascon_sbox, là S-box 5 bit áp song song trên 64 vị trí bit.

Lớp tuyến tính ascon_linear, gồm phép xoay phải kết hợp XOR.

Bộ đếm vòng round_idx tra hằng số vòng RC bằng khối case thay vì dùng mảng lớn. Cách này nhằm tránh việc công cụ tổng hợp suy ra Block RAM, một ràng buộc đặt ra ngay từ đầu cho thiết kế.

Đường XOR dữ liệu vào chỉ tác động lên hai từ rate là S0 và S1. Riêng với khối bản rõ đầu tiên, thiết kế XOR thêm bit phân tách miền vào S4. Đường xuất dữ liệu phân biệt hai chế độ: khi mã hóa thì xuất phần rate sau khi đã XOR, còn khi giải mã thì xuất phần rate trước khi bị ghi đè.

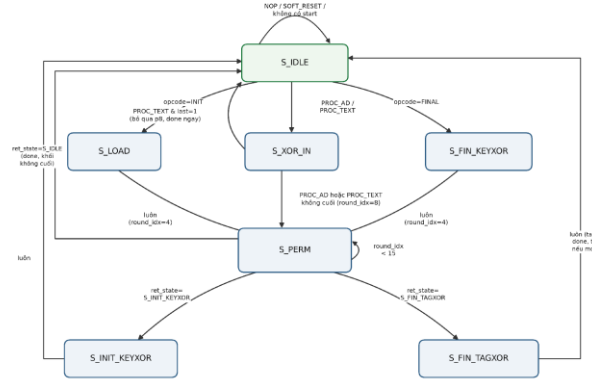


Hình 3 — Datapath kiến trúc một vòng hoán vị mỗi chu kỳ.

Hình 4 trình bày máy trạng thái ascon_aead_fsm với bảy trạng thái. Trạng thái S_PERM được dùng chung cho cả hoán vị p8 lẫn p12, nhằm tránh nhân đôi logic điều khiển. Hai hoán vị chi khác nhau ở giá trị nạp cho round_idx, là 4 với p12 và 8 với p8, cùng với trạng thái quay về ret_state sau khi hoán vị kết thúc.

Bảng trạng thái đặc tả rõ một quy tắc dễ hiện thực sai: p8 luôn chạy sau mỗi khối xử lý AD, kể cả khối AD cuối cùng, nhưng không bao giờ chạy sau khối bản rõ cuối cùng.

Quá trình kiểm chứng đã phát hiện và khắc phục một lỗi bảo mật ở tầng FSM này. Khi giải mã, tín hiệu dout_valid từng được bật vô điều kiện ngay tại lệnh xử lý khối bản rõ cuối, kể cả trong trường hợp tag sẽ không khớp. Hệ quả là bản rõ có thể bị lộ ra ngoài trước khi quá trình xác thực hoàn tất, vi phạm nguyên tắc cơ bản của mã hóa xác thực. Bản sửa bổ sung một thanh ghi giữ lại bản rõ khối cuối cho tới khi kiểm tra tag xong, và chỉ xuất dữ liệu ra khi tag khớp.



Hình 4 — Máy trạng thái điều khiển AEAD.

Giao diện thanh ghi phơi ra qua APB gồm hai thanh ghi điều khiển tại offset 0x00 và 0x04:

CMD — thanh ghi lệnh, chứa opcode 3 bit, cờ last, cờ chọn chế độ mã hóa hoặc giải mã, và số byte hợp lệ valid_bytes.

STATUS — thanh ghi trạng thái, chứa các cờ busy, done, dout_valid, tag_valid, tag_fail và din_full.

Các offset còn lại chứa sáu nhóm thanh ghi dữ liệu rộng 128 bit là KEY, NONCE, DIN, DOUT, TAG và TAGIN, mỗi nhóm gồm bốn từ 32 bit. Do bề rộng bus APB là 32 bit, mỗi khối dữ liệu 128 bit cần bốn lần ghi hoặc bốn lần đọc để truyền qua bus.

Riêng nhóm KEY chỉ cho phép ghi, thao tác đọc luôn trả về giá trị 0. Đây là biện pháp bảo mật bắt buộc với một IP mật mã, nhằm ngăn khóa bị đọc ngược ra ngoài qua bus.

Ba phương án vi kiến trúc được khảo sát khác nhau ở tham số ROUNDS_PER_CYCLE, ký hiệu là R, nhận các giá trị 1, 2 và 4. Tham số này quy định số vòng ascon_round được nối tiếp tổ hợp thuần túy và thực thi trong một chu kỳ đồng hồ, không chen thanh ghi trung gian.

Số chu kỳ lõi cần để xử lý A khối AD và T khối bản rõ hoặc bản mã tuân theo công thức:

$$N_{\text{cycle_R}}(A, T) = (3 + 24/R) + (1 + 8/R) \cdot (A + T)$$

Công thức này suy trực tiếp từ việc mỗi lần chạy p12 tốn 12/R chu kỳ và mỗi lần chạy p8 tốn 8/R chu kỳ. Phần chu kỳ điều khiển cố định, gồm nạp trạng thái, XOR dữ liệu và XOR khóa, không co lại theo R. Đó là lý do số chu kỳ không giảm đúng theo tỉ lệ 1/R.

III. KẾT QUẢ

A. Kiểm chứng chức năng

Phần này trình bày kết quả kiểm chứng chức năng dựa trên bộ test vector KAT chính thức của NIST [3], kết quả tổng hợp và PPA cho ba phương án vi kiến trúc và ba dòng FPGA, so sánh với các công trình liên quan, cùng kết quả đo công suất và phân tích timing tĩnh sau khi đặt chỗ và đi dây.

Bảng 1 — Kết quả kiểm chứng chức năng và timing tĩnh

Hạng mục kiểm chứng	Nội dung	Kết quả
Mô hình tham chiếu Python	1089 vector KAT chính thức của NIST	PASSED 1089/1089
Unit RTL: ascon_sbox, ascon_linear, ascon_round, ascon_perm	Số khớp từng vòng hoán vị p12 và p8 với mô hình tham chiếu, trên cả ba kiến trúc	PASSED 20/20 mỗi test, cả ba kiến trúc
RTL AEAD, mã hóa và giải mã (tb_aead.v)	1089 vector KAT, trên cả ba kiến trúc RPC = 1, 2 và 4	encrypt 1089/1089; decrypt 1089/1089
RTL AEAD, lật bit (tb_aead.v)	Lật một bit trong ciphertext, tag hoặc AD; xác nhận cờ tag_fail và việc chặn DOUT khối cuối	ct_flip 56/56; tag_flip 58/58; ad_flip 56/56
APB slave và protocol checker (tb_apb.v)	Bảy test case biên, 1089 vector truyền qua APB, và bộ kiểm tra giao thức APB thuần Verilog	Tất cả PASSED; 0 vi phạm giao thức
Gate-level mức chức năng (post-route, RPC=1)	20 vector KAT chọn lọc trên netlist funcsim, không chú thích SDF	PASSED 20/20
Timing tĩnh (post-route, RPC=1, chu kỳ 5.5 ns)	report_timing_summary trên netlist đã route	WNS = +0.013 ns; WHS = +0.109 ns; 0 vi phạm

Bảy test case biên của bộ kiểm tra APB gồm: đọc thanh ghi KEY và xác nhận trả về 0, đọc khi bộ đệm rỗng, truy cập địa chỉ không được ánh xạ, reset giữa chừng một thao tác, ghi khi lỗi đang bận, ghi đè thanh ghi DIN, và hai giao dịch liên tiếp không có chu kỳ nghỉ.

Toàn bộ các cấp độ kiểm chứng ở Bảng 1 đều đạt PASSED. Đồ án đáp ứng bốn trong năm tiêu chí nghiệm thu bắt buộc:

- Pass 100% test vector NIST ở mô phỏng RTL.
- Pass test vector ở mô phỏng gate-level mức chức năng, sau đặt chỗ và đi dây.
- Không vi phạm timing sau implementation.

Báo cáo PPA cho từ hai phương án vi kiến trúc trở lên.

Tiêu chí thứ năm, mô phỏng gate-level có chú thích SDF, chưa đạt được trọn vẹn. Bộ mô phỏng xsim báo lỗi annotate SDF trên toàn bộ thực thể FDCE, do thư viện mô phỏng unisims_ver cài kèm tại máy build thiếu specify-block cho biến thể FDCE_default. Đây là một hạn chế còn tồn đọng, được trình bày chi tiết ở mục IV.

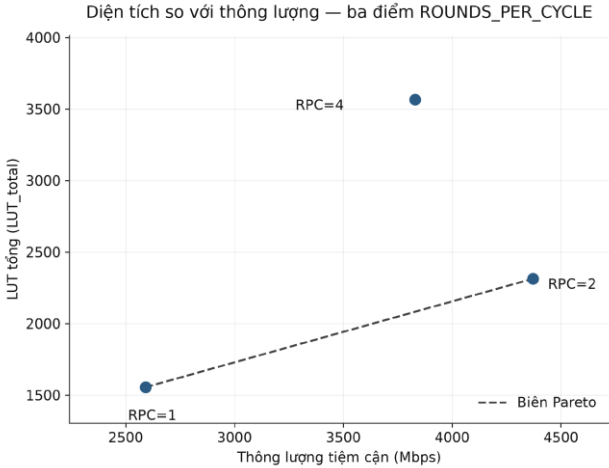
B. PPA theo phương án vi kiến trúc

Bảng 2 trình bày kết quả tổng hợp Out-of-Context và quét tần số hoạt động tối đa (Fmax) cho ba phương án vi kiến trúc, trên cùng một thiết bị đích là Artix-7 xc7a35tcpg236-1 với speed grade -1.

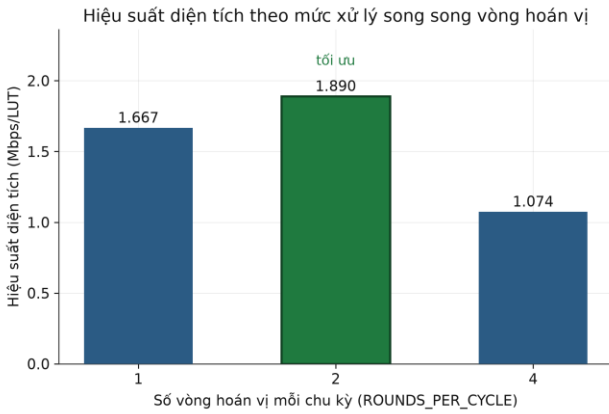
Bảng 2 — PPA ba phương án vi kiến trúc ROUNDS_PER_CYCLE (Artix-7, xc7a35tcpg236-1, -1)

Kiến trúc	Chu kỳ chọn / Fmax (MHz)	LUT core/tổng	FF core/tổng	Chu kỳ /khối	Thông lượng tiệm cận (Mbps)	Mbps /LUT
RPC=1	5.5 ns / 182.25	1437 / 1555	732 / 1517	9	2592.00	1.6669
RPC=2	6.0 ns / 170.74	2195 / 2313	731 / 1516	5	4370.94	1.8897
RPC=4	11.2 ns / 89.77	3447 / 3565	730 / 1515	3	3830.19	1.0743

Hiệu suất diện tích (Mbps/LUT) tăng từ RPC=1 sang RPC=2 (+13.4%) rồi đảo chiều giảm mạnh ở RPC=4 (-43.2% so với RPC=2, thấp hơn cả RPC=1) — đây không phải hiện tượng bão hòa dần mà là đảo chiều thật sự, vì logic tổ hợp bốn ascon_round nối tiếp vừa tốn LUT nhiều hơn tuyến tính vừa kéo Fmax giảm gần một nửa, lần át hoàn toàn phần lợi từ số chu kỳ mỗi khối thấp hơn. Xét trên cả hai trục diện tích và thông lượng tuyệt đối, RPC=2 là điểm tối ưu Pareto trong ba điểm đã đo — RPC=4 bị RPC=2 lấn át hoàn toàn (thông lượng thấp hơn và LUT cao hơn), thể hiện rõ ở Hình 5 và Hình 6.



Hình 5 — Diện tích so với thông lượng tiệm cận của ba cấu hình $ROUNDS_PER_CYCLE$, với $RPC=1$ và $RPC=2$ nằm trên biên Pareto.

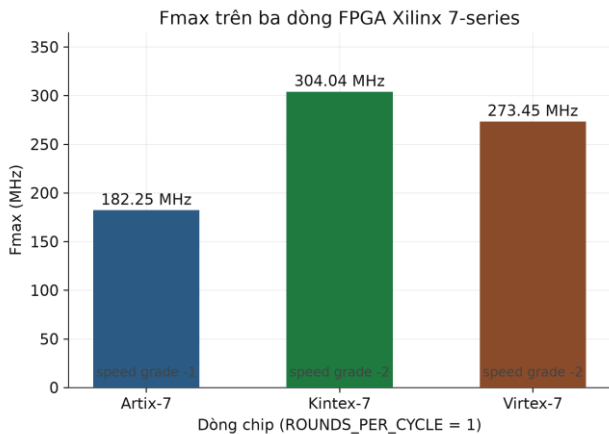


Hình 6 — Hiệu suất diện tích của ba cấu hình $ROUNDS_PER_CYCLE$, đạt đỉnh tại $RPC=2$.

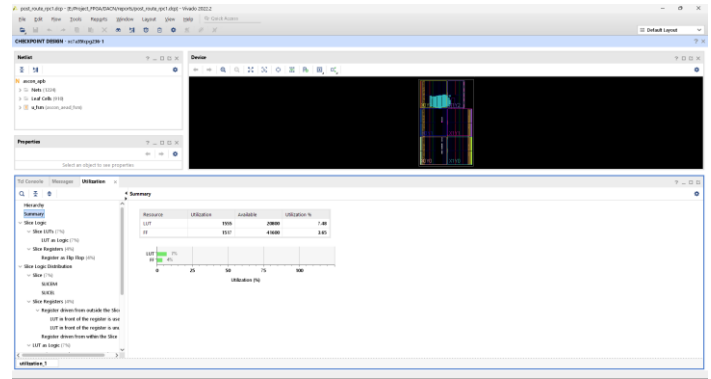
Bảng 3 — PPA đa thiết bị ($ROUNDS_PER_CYCLE=1$)

Dòng chip	Part	Speed grade	Fmax (MHz)	LUT core/tổng	FF tổng	Thông lượng (Mbps)	Mbps /LUT
Artix-7	xc7a35tcpg236-1	-1	182.25	1437 / 1555	1517	2592.00	1.6669
Kintex-7	xc7k325tffg900-2	-2	304.04	1439 / 1557	1517	4324.12	2.7772
Virtex-7	xc7vx485tffg1761-2	-2	273.45	1440 / 1558	1517	3889.07	2.4962

LUT_total chỉ dao động trong khoảng 1555–1558 (chênh lệch dưới 0.2%) và FF_total giống hệt nhau tuyệt đối (1517) trên cả ba dòng chip — vì cả ba đều dùng chung kiến trúc slice 7-series (LUT6 hai đầu ra, hai FF trên mỗi LUT, cùng nguyên thủy CARRY4) và thiết kế không dùng Block RAM hay DSP, nên toàn bộ chênh lệch PPA giữa ba dòng chip nằm ở Fmax, minh họa ở Hình 8. Đường tới hạn thật trên cả ba dòng chip đều là cùng một chuỗi CARRY4 phát sinh từ phép so sánh tag 128 bit trong ascon_aead_fsm, không phải round-datapath của ascon_perm — một phát hiện bất ngờ vì phép so sánh tag không có toán tử số học nào theo đặc tả thuật toán.



Hình 8 — Fmax trên ba dòng FPGA Xilinx 7-series cùng RTL, cùng ràng buộc. Artix-7 đo ở speed grade -1 (bảo thủ nhất trong họ); Kintex-7 và Virtex-7 đo ở -2.



Hình 7 — Báo cáo Utilization sau đặt chỗ và đi dây cho cấu hình $ROUNDS_PER_CYCLE=1$ trên xc7a35tcpg236-1.

C. PPA đa thiết bị

Bảng 3 khảo sát cùng một RTL và cùng ràng buộc ($ROUNDS_PER_CYCLE=1$) trên ba dòng FPGA Xilinx 7-series khác nhau.

D. Công suất và timing tĩnh

Công suất được đo bằng report_power trên hoạt động chuyển mạch thực, với dữ liệu SAIF trích từ mô phỏng gate-level mức chức năng. Phép đo thực hiện ở cấu hình $RPC=1$ trên Artix-7, chu kỳ đồng hồ 5.5 ns. Tổng công suất on-chip là 91 mW, gồm 23 mW công suất động và 68 mW công suất tĩnh. Trong phần công suất động, khối ascon_aead_fsm chiếm 18 mW.

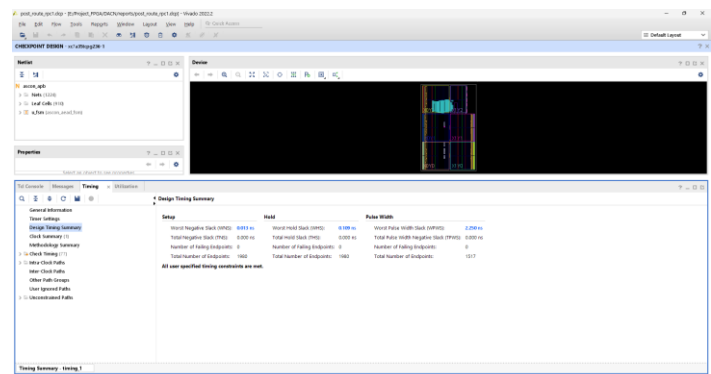
Về timing tĩnh, report_timing_summary trên netlist đã route tại chu kỳ 5.5 ns cho ba chỉ số sau:

WNS = +0.013 ns, không có endpoint nào vi phạm setup.

WHS = +0.109 ns, không có endpoint nào vi phạm hold.

WPWS = +2.250 ns, không có endpoint nào vi phạm độ rộng xung.

Kết quả này đáp ứng tiêu chí không vi phạm timing sau implementation.



Hình 9 — Báo cáo Timing Summary sau đặt chỗ và đi dây cho cấu hình $ROUNDS_PER_CYCLE=1$ tại chu kỳ 5.5 ns.

E. So sánh với công trình liên quan

Bảng 4 đối chiếu kết quả trên Artix-7 với hai công trình liên quan gần đây hiện thực họ thuật toán Ascon trên FPGA. Cần lưu

Bảng 4 — So sánh quốc tế trên Artix-7

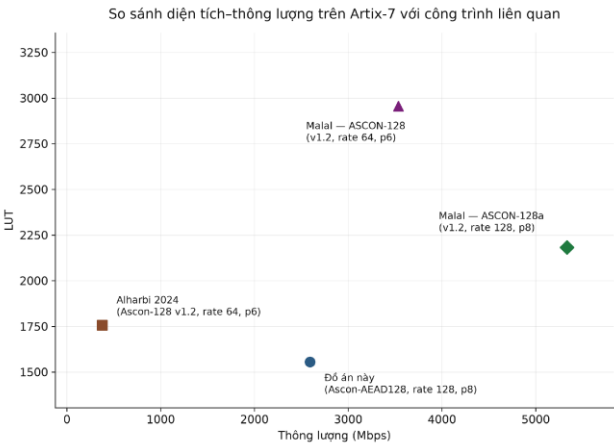
Công trình	Biến thể Ascon	FPGA (speed grade)	LUT	Fmax (MHz)	Thông lượng (Mbps)	Mbps /LUT
Đồ án này	Ascon-AEAD128 (SP800-232, rate128, p8)	xc7a35tcpg236-1 (-1)	1555	182.25	2592.00	1.667
Alharbi 2024 [4]	Ascon-128 v1.2 (rate64, p6)	xc7a100tcsg324-3 (-3)	1756	317	376	0.214
Malal [5]	ASCON-128 v1.2 (rate64, p6)	xc7a200tfbg676-2 (-2)	2957	55.25	3535.91	1.19
Malal [5]	ASCON-128a v1.2 (rate128, p8)	xc7a200tfbg676-2 (-2)	2183	83.33	5333.30	2.44

So với Alharbi và cộng sự [4], vốn cùng lớp kiến trúc iterative một vòng hoán vị mỗi chu kỳ như đồ án này, đồ án đạt hiệu suất diện tích cao hơn khoảng 7.8 lần, dù sử dụng speed grade chậm nhất là -1 trong khi Alharbi dùng -3. Nguyên nhân chính không nằm ở vi kiến trúc vượt trội, mà ở chỗ rate 128 bit cho thông lượng mỗi khối gấp đôi so với Ascon-128 v1.2, ở cùng số chu kỳ hoán vị p8 so với p6. Cần nêu rõ điều này để tránh ngộ nhận về nguồn gốc của lợi thế.

Phép so sánh công bằng nhất về cấu trúc dữ liệu là với biến thể Ascon-128a của Malal [5], vì biến thể này cũng dùng rate 128 bit. Ở đây Malal đạt hiệu suất diện tích cao hơn đồ án, 2.44 so với 1.667 Mbps trên mỗi LUT trên Artix-7. Lợi thế đó đến từ kiến trúc xử lý bốn vòng hoán vị song song mỗi chu kỳ, đánh đổi bằng Fmax thấp hơn nhiều, chỉ 83 MHz so với 182 MHz của đồ án. Đây là hai điểm khác nhau trên cùng một không gian đánh đổi mà đồ án đã khảo sát ở Bảng 2.

Ngoài các công trình cùng họ Xilinx 7-series nói trên, Mohamed và cộng sự [6] hiện thực Ascon-128 v1.2 bằng kiến trúc serial hóa trên FPGA Lattice iCE40, đạt 582 LUT. Kết quả này được xem như một mốc tham khảo cận dưới về diện tích, nhưng không đưa vào Bảng 4 vì khác cả kiến trúc lẫn họ linh kiện.

Hình 10 minh họa kết quả so sánh.



ý ngay: cả hai công trình đối chiếu hiện thực Ascon-128 bản đề trình v1.2 (rate 64 hoặc 128 bit tùy biến thể, hoán vị trung gian p6 hoặc p8, big-endian) — không phải Ascon-AEAD128 theo NIST SP 800-232 như đồ án này — nên số liệu chỉ mang tính định vị tương đối trong họ Ascon, không phải so sánh ngang hàng tuyệt đối.

Hình 10 — So sánh diện tích và thông lượng trên Artix-7 với ba cấu hình từ hai công trình liên quan. Các công trình đối chiếu hiện thực Ascon-128 bản v1.2, không phải Ascon-AEAD128 theo NIST SP 800-232, nên số liệu chỉ mang tính định vị tương đối.

F. Demo hệ thống

Đồ án đã xây dựng thêm một hệ thống demo giao tiếp qua UART, nhắm tới board Genesys 2 dùng chip Kintex-7. Hệ thống gồm RTL cho khối uart_rx và uart_tx, một APB master điều khiển bằng lệnh nhận qua UART, cùng testbench ở mức hệ thống.

Mô phỏng testbench này bằng Icarus Verilog với mô hình chức năng bus UART đã pass cả hai vector kiểm tra, gồm KAT Count=1 và Count=166. Đồ án cũng đã sinh thành công bitstream cho đúng thiết bị của board. Tuy nhiên do không tiếp cận được phần cứng, việc nạp và đo trực tiếp chưa thực hiện được. Nội dung này được thảo luận ở mục IV.

IV. KẾT LUẬN

Đồ án đã hoàn thành một lõi IP phần cứng hiện thực đúng đặc tả Ascon-AEAD128 theo NIST SP 800-232. Đây là chuẩn còn rất mới, khác biệt cấu trúc rõ rệt so với các công trình FPGA công bố trước đó vốn hiện thực bản v1.2 cũ hơn. Lõi được kiểm chứng qua bốn tầng:

Mô hình tham chiếu Python đối chiếu 100% test vector KAT chính thức của NIST.

RTL kiểm chứng bằng cả test dương lẫn các trường hợp lật bit.

Một bộ kiểm tra giao thức APB thuần Verilog độc lập.

Xác nhận chức năng lần nữa trên chính netlist sau đặt chỗ và đi dây.

Đồ án cũng vượt phạm vi tối thiểu bằng cách khảo sát ba phương án vi kiến trúc, so với yêu cầu tối thiểu là hai, và ba dòng FPGA Xilinx 7-series. Quá trình khảo sát rút ra một kết quả phân trực giác đáng chú ý: $ROUNDS_PER_CYCLE$ bằng 2 mới là điểm tối ưu Pareto, chứ không phải $RPC=4$ dù phương án này xử

lý song song nhiều vòng hoán vị hơn. Ngoài ra, quá trình kiểm chứng đã phát hiện và khắc phục một lỗi bảo mật thật sự, đó là rò rỉ bản rõ khỏi cuối trước khi kiểm tra tag.

Tại thời điểm báo cáo còn hai hạn chế chính.

Thứ nhất, mô phỏng gate-level có chú thích SDF chưa thực hiện được. Thư viện mô phỏng unisims_ver cài kèm Vivado 2022.2 tại máy build thiếu specify-block timing cho biến thể FDCE_default, khiến xsim không annotate được delay. Phương án dự phòng đã áp dụng là tách phân tích timing, thực hiện tính bằng report_timing_summary trên chính netlist đã route, ra khỏi phần kiểm chứng chức năng và đo công suất, vốn dùng mô phỏng gate-level không SDF kết hợp SAIF. Đây vẫn là bằng chứng gián tiếp, không thay thế hoàn toàn cho một lần mô phỏng có trễ thật.

Thứ hai, demo trên board chưa được xác nhận trên phần cứng vật lý. Đồ án đã có đầy đủ RTL, ràng buộc thời gian XDC và testbench hệ thống cho demo UART, đồng thời đã sinh thành công bitstream cho đúng thiết bị của board Genesys 2. Tuy nhiên do không tiếp cận được phần cứng, việc đo và xác nhận trực tiếp chưa thực hiện được. Nội dung này thuộc phạm vi mở rộng, không bắt buộc, nhưng vẫn là khoảng trống giữa kết quả mô phỏng và bằng chứng phần cứng thật.

Ba hướng phát triển được xác định như sau.

Một, dựng lại thư viện mô phỏng xsim bằng compile_simlib để chạy được mô phỏng gate-level có chú thích SDF đầy đủ. Lần thử trước phải dùng vì Vivado không giới hạn được biên dịch theo đúng ngôn ngữ Verilog, và bước trích xuất dữ liệu từ IP repository kéo dài quá lâu.

Hai, nạp bitstream lên board Genesys 2 và đo trực tiếp giao tiếp UART end-to-end, hoàn thiện nốt nội dung mở rộng vốn đã có sẵn RTL và mô phỏng hệ thống.

Ba, tiếp tục điều tra chuỗi CARRY4 phát sinh từ phép so sánh tag 128 bit. Nguyên nhân gốc chưa được giải quyết triệt để: các đôn bẫy đã thử ở mức RTL và tổng hợp, gồm viết lại bằng case, dùng thuộc tính use_carry và dont_touch, tắt resource_sharing, đều không thay đổi được lựa chọn công nghệ ảnh xạ của Vivado. Hướng khả thi nhất còn lại là tách vật lý phép so sánh tag ra một module riêng, nhằm phá vỡ cơ hội chia sẻ tài nguyên carry-chain.

Ngoài ba hướng trên, việc tách bạch ảnh hưởng của speed grade khỏi ảnh hưởng của dòng chip lên Fmax hiện mới chỉ suy luận gián tiếp, thông qua cặp Kintex-7 và Virtex-7 cùng speed grade -2. Đo thêm một điểm cùng die nhưng khác speed grade, chẳng hạn xc7a35tcpg236-2, sẽ giúp kết luận này chặt chẽ hơn.

V. TÀI LIỆU THAM KHẢO

- [1] [1] National Institute of Standards and Technology, "Ascon-Based Lightweight Cryptography Standards for Constrained Devices: Authenticated Encryption, Hash, and Extendable Output Functions," NIST SP 800-232, Aug. 2025.
- [2] [2] Arm Limited, "AMBA APB Protocol Specification," ARM IHI 0024E (Issue E), Feb. 2023.
- [3] [3] National Institute of Standards and Technology, "Lightweight Cryptography Project — Ascon Known-Answer Tests," [Online]. Available: <https://csrc.nist.gov/projects/lightweight-cryptography>
- [4] [4] A. R. Alharbi, A. Aljaedi, A. Aljuhni, M. K. Alghuson, H. Aldawood, and S. S. Jamal, "Evaluating Ascon Hardware on 7-Series FPGA Devices," IEEE Access, vol. 12, pp. 149076–149089, 2024, doi: 10.1109/ACCESS.2024.3471694.
- [5] [5] A. Malal, "High-Performance FPGA Implementations of Lightweight ASCON-128 and ASCON-128a With Enhanced Throughput-to-Area Efficiency," in Proc. 2024 17th Int. Conf. Inf. Secur. Cryptol. (ISCTürkiye), Ankara, Türkiye, 2024, pp. 1–7, doi: 10.1109/ISCTrkiye64784.2024.10779273.
- [6] [6] H. A. Mohamed, M. Taher, H. M. Shousha, Z. Mohsen, M. M. Abdelrazik, Y. Ismail, and A. Saeed, "An efficient serialized hardware implementation of the ASCON algorithm," Scientific Reports, vol. 16, art. no. 23197, 2026, doi: 10.1038/s41598-026-63223-6.